Trabajo Práctico Nº 4

Mecanismos de sincronización: semáforos binarios

1. Dado los siguientes códigos:

ContadorSincronico.java

```java
public class ContadorSincronico {
    private int valor = 0;

    public synchronized void incrementar() {
        valor++;
    }

    public void decrementar() {
        valor--;
    }

    public synchronized int getValor() {
        return valor;
    }
}
```

ContadorObjetoSincronico.java

```java
public class ContadorObjetoSincronico {
    private int valor = 0;

    public void incrementar() {
        synchronized ((Integer) valor) {
            valor++;
        }
    }

    public void decrementar() {
        synchronized (this) {
            valor--;
        }
    }

    public synchronized int getValor() {
        return valor;
    }
}
```

a) Verifique el funcionamiento del código de la clase "`ContadorSincronico`".

b) Verifique el funcionamiento del código de la clase "`ContadorObjetoSincronico`".

c) Realice las correcciones que crea convenientes para mejorar el funcionamiento del programa.

d) Compare el uso de métodos sincronizados con objetos sincronizados, dando ventajas y desventajas en cada caso.

2. Considere un programa en el que hay cuatro procesos ejecutando cada uno una función distinta. Para su sincronización, se disponen cuatro semáforos binarios (sem1, sem2, sem3 y sem4).

```
Proceso P1
    adquirir(sem2);
    operaciones_P1;
    liberar(sem1);
end
```

```
Proceso P2
    adquirir(sem3);
    operaciones_P2;
    liberar(sem2);
end
```

```
Proceso P3
    adquirir(sem1);
    operaciones_P3;
    liberar(sem3);
    liberar(sem4);
end
```

```
Proceso P4
    adquirir(sem4);
    operaciones_P4;
end
```

```
inicializa(sem1, 0);
inicializa(sem2, 1);
inicializa(sem3, 0);
inicializa(sem4, 0);
```

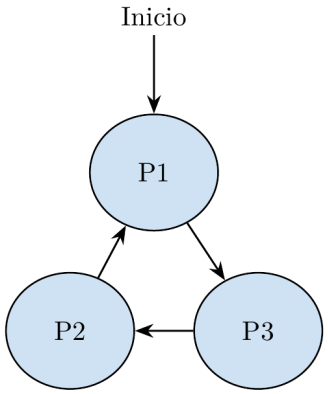
a) Según la inicialización dada, realizar un diagrama mostrando el orden de ejecución de las operaciones de cada proceso, indicando claramente qué operaciones se ejecutan antes que otras y cuáles se ejecutan de forma concurrente.

b) ¿Qué sucede si el semáforo sem2 se inicializa en 0?

c) ¿Qué sucede si el semáforo sem1 se inicializa en 1?

3. Implementar en java un programa para lograr la sincronización de tres procesos (P1, P2 y P3) de forma que se establezca el orden de ejecución indicado en la imagen. De esta manera,

primero se ejecuta P1, cuando finaliza se puede ejecutar P3, y al terminar se ejecuta P2. Una vez que concluya, se puede ejecutar P1 y continúa el ciclo infinitamente.



4. Gestión de Impresoras en un Centro de Copiado

Imagina un centro de copiado con múltiples impresoras disponibles para los clientes. Los clientes llegan con sus documentos para imprimir y utilizan las impresoras disponibles. Para garantizar un uso eficiente de las impresoras, se implementará un sistema de gestión llamado "GestorImpresoras". Cada impresora tiene un estado (ocupada o disponible) y se representa como un recurso compartido. Los clientes se modelan como hilos que llegan al centro de copiado y desean imprimir sus documentos.

El GestorImpresoras se encarga de regular el acceso a las impresoras de la siguiente manera:

- Si hay alguna impresora disponible, el cliente que llega puede imprimir su documento de inmediato.
- Si todas las impresoras están ocupadas, el cliente debe esperar hasta que una impresora esté disponible.
- Cada impresión lleva un cierto tiempo, simulado con un retraso.

Diseña un programa en el que:

a) Los clientes llegan al centro de copiado y solicitan imprimir un documento.

b) El gestor de impresoras realice la acción que corresponda según las indicaciones previamente establecidas.

c) Se debe utilizar semáforos para garantizar el acceso sincronizado a las impresoras.

5. Mejorar el código del punto anterior para que ahora, el centro de impresión cuente con dos tipos distintos de impresoras para dar servicio a los usuarios: impresoras de tipo A e impresoras de tipo B. El número de impresoras de cada tipo es limitado (numA impresoras de tipo A y numB impresoras de tipo B). Para imprimir un trabajo en una impresora de un tipo determinado, el cliente debe hacer la solicitud de impresión indicando el tipo requerido,

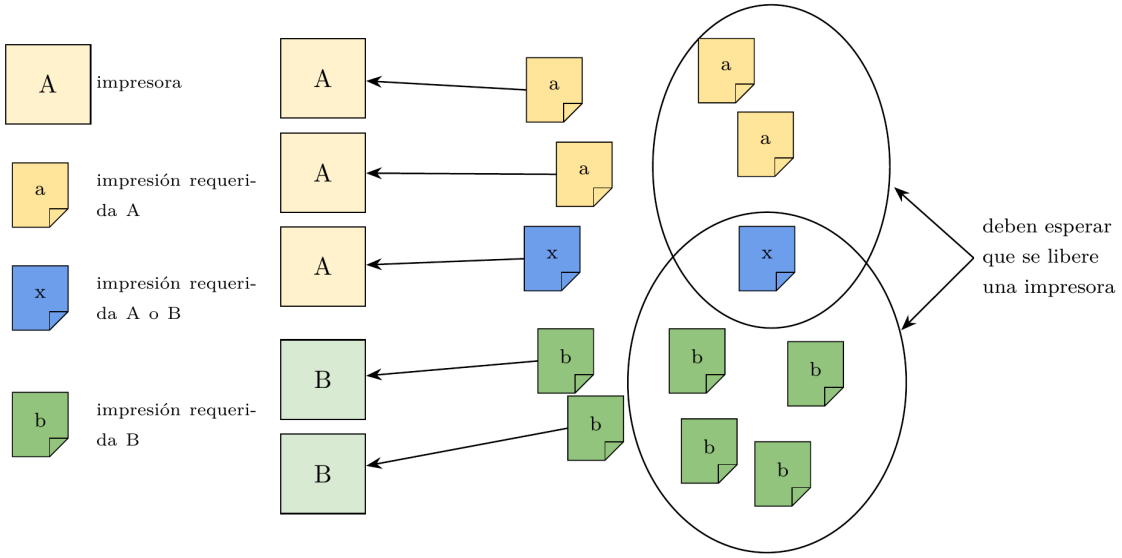
considerando tres grupos distintos:

- Los que requieren una impresora de tipo A.
- Los que requieren una impresora de tipo B.
- Los que pueden utilizar una impresora de uno cualquiera de los dos tipos (indicado con la letra X).

Los clientes solicitan la impresión de sus documentos según el tipo deseado, y el gestor de impresoras es el encargado de asignar a cada cliente una impresora compatible. Se deben tener en cuenta las mismas restricciones que en el ejercicio anterior, pero teniendo en cuenta los tipos.

Cuando un usuario quiera imprimir un trabajo deberá hacerlo sobre una impresora compatible con él y que no esté siendo utilizada por otro usuario. En otro caso el proceso deberá esperar.

Ejemplo: Tenemos 3 impresoras de tipo A y dos impresoras de tipo B. Cada una puede imprimir un trabajo a la vez.



6. Taxista

Considere los siguientes dos hilos: un taxista y usted. Usted necesita un taxi para llegar a su destino y el taxista necesita que un pasajero tome el taxi para cobrar por su trabajo.

De esta manera, cada uno tiene una tarea: usted espera tomar el taxi y, una vez dentro, debe viajar hasta que el taxista le notifique que ha llegado a su destino. Esto último se realiza ya que resultaría molesto para usted y para el taxista preguntar cada dos segundos "¿Ya llegamos a destino?".

Entre distintos viajes, el taxista quiere dormir en el taxi hasta que otro pasajero necesite ser conducido hasta algún lugar, no desea despertarse de su siesta cada cinco minutos para ver si arribó un pasajero. Por lo tanto, ambos hilos preferirían realizar su trabajo de la manera más relajada posible.

Usted y el taxista necesitan alguna forma de comunicar sus necesidades al otro. Mientras usted está ocupado caminando por la calle buscando un taxi, el taxista está durmiendo plácidamente en la cabina. Cuando usted le avisa que quiere tomar el taxi, él se despierta y comienza a conducir. Cuando usted arriba a su destino, el taxista le notifica que ha llegado y continúa con su trabajo. El taxista debe ahora esperar y dormir nuevamente una siesta hasta que el próximo pasajero llegue.

Se solicita:

a) Realizar un dibujo/esquema/diagrama para entender cómo funciona la sincronización entre los hilos, el flujo específico de cada uno y del programa en su completitud.

b) Diseñar una solución de comunicación de los hilos que modele dicha situación.

7. **Retomar el ejercicio 9 del tp anterior** (autos, camión y surtidor de combustible). Implementar utilizando semáforos binarios.

8. **Los Pollos Hermanos**

   Una pequeña empresa de k empleados dispone de un pequeño lugar donde los empleados pueden tomar algo rápido (por ejemplo, un café y una porción de pollo frito). La pequeña confitería es atendida por **un mozo** y tiene espacio para **un empleado**.

   Cuando un empleado desea servirse algo se acerca al lugar, y si el espacio está libre se sienta en la mesa que está disponible. Si el lugar ya se encuentra ocupado, se queda esperando a que se libere espacio dentro de la confitería.

   Una vez dentro, le indica al mozo que está allí para que le sirvan. Elige lo que desea de una pequeña lista de opciones, se la indica al mozo, y espera pacientemente que le sirvan. Cuando le han servido, el mozo le indica que puede empezar a comer. El empleado come y, cuando termina, deja la silla libre, agradece al mozo y vuelve a trabajar.

   Por su parte el mozo, cuando no hay empleados que atender, aprovecha a disfrutar de su hobbie de inventar nuevas versiones de pollo, hasta que llegue otro empleado.

   a) Escriba un programa que ejecute las funciones descritas utilizando los mecanismos de sincronización que considere necesarios. Debe existir un hilo para el mozo y un hilo para cada empleado. La solución debe estar libre de bloqueos. La solución debe maximizar la concurrencia de los hilos.

9. **Fábrica de producción**

   Imaginemos una fábrica de producción con dos líneas de ensamblaje, una para productos eléctricos y otra para productos mecánicos. Para coordinar el flujo de producción, se solicita implementar un sistema de gestión llamado **CentroProduccion**.

   Cada línea de ensamblaje tiene su propio equipo de trabajadores que ensamblan los productos. Los productos eléctricos llegan desde el norte, mientras que los productos mecánicos llegan desde el oeste. Cada línea tiene una capacidad máxima de productos que pueden contener a

la vez. En la entrada de cada línea hay sensores que detectan la llegada de un producto y también hay sensores que indican cuando un producto sale de la línea de ensamblaje. Ambas líneas desembocan en una única intersección de salida, por la que los productos abandonan la fábrica.

El **CentroProduccion** se encarga de regular el flujo de los productos entre las dos líneas de ensamblaje. Cada **producto** se representa como un hilo que invoca los métodos *llegaElectrico()* o *llegaMecanico()* cuando llega al inicio de su línea correspondiente.

Si la línea de ensamblaje tiene capacidad, el producto entra inmediatamente y ocupa un lugar en ella. En cambio, si esta llena debe esperar en la entrada a que se libere un lugar. Cuando el producto logra entrar a la linea requiere un cierto tiempo para ser ensamblado. Una vez ensamblado, el producto invoca el método *saleElectrico()* o *saleMecanico()* según su tipo, para indicar que ha terminado y abandonar la línea a través de la intersección.

La intersección tiene habilitado un solo lado por vez, de modo que únicamente pueden salir los productos del tipo que corresponde al lado activo, uno por vez, mientras que los del otro tipo (aunque ya estén ensamblados) deben esperar su turno sin liberar el lugar que ocupan en su línea.

Además, hay un hilo de **control** que llama al método *cambiaLineas()* cada cierto tiempo para cambiar la configuración de las líneas de ensamblaje, alternando cuál de los dos lados de la intersección está habilitado.

En resumen, el CentroProduccion simula la gestión de la producción en una fábrica con dos líneas de ensamblaje, regulando la entrada de productos según la capacidad de cada línea y la salida de los mismos por una intersección compartida que alterna entre ambos tipos de producto.

10. Barbero dormilón

En una barbería trabaja un barbero que dispone de un único sillón de atención y de varias sillas de espera. Cuando no hay clientes, el barbero se duerme en su sillón hasta que llegue alguno. Cuando llega un cliente pueden darse tres situaciones:

- Si el barbero está durmiendo, el cliente lo despierta y pasa a ser atendido.
- Si el barbero está atendiendo a otro cliente y hay sillas de espera libres, el cliente ocupa una de ellas y espera su turno.
- Si el barbero está ocupado y todas las sillas de espera están ocupadas, el cliente se retira y no regresa.

Al terminar de atender a un cliente, el barbero toma al siguiente que está esperando, y si no hay ninguno vuelve a dormirse.

Implementar las siguientes versiones del problema:

a) Sin sillas de espera. La barbería cuenta únicamente con el sillón de atención. Si está ocupado, el cliente que llega se retira.

b) **Con sala de espera.** La barbería cuenta con n sillas de espera, según el comportamiento descrito anteriormente.

c) **Con varios barberos.** En la barbería trabajan m barberos, cada uno con su propio sillón, pero compartiendo las sillas de espera. Un cliente que es atendido por cualquier barbero que esté libre.